

Brush

1. Experiment Objective

Use MediaPipe hand tracking to implement a virtual paint brush: draw on the camera image with the index fingertip, with support for selecting multiple colors and erasing.

2. Experiment Principle

1. Key Terms

Keyword	Description
Virtual paint brush	Tracks the index fingertip coordinates to draw continuous lines on the canvas, simulating a real paint brush experience
Drawing canvas	A separate image layer with the same size as the source frame; brush strokes are drawn on the canvas and then blended with the original image
Color selection	A preset color bar at the top of the frame; touching the corresponding region with the fingertip switches the brush color

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start the virtual paint brush
```

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripherals	ESP32 camera

2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

3. Start the virtual paint brush (Terminal 2)

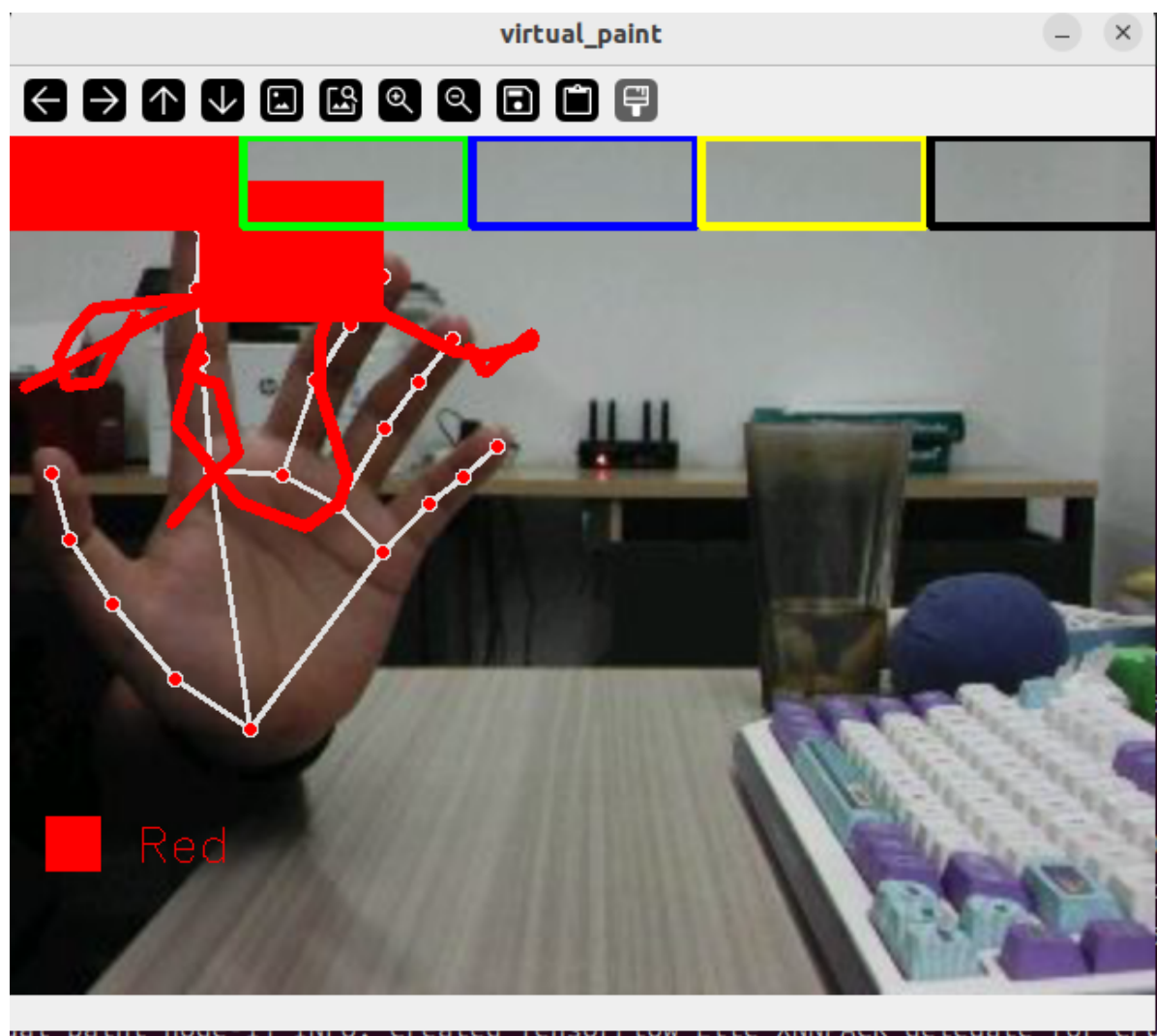
```
ros2 launch microros_v2_mediapipe_pkg virtual_paint.launch.py
```

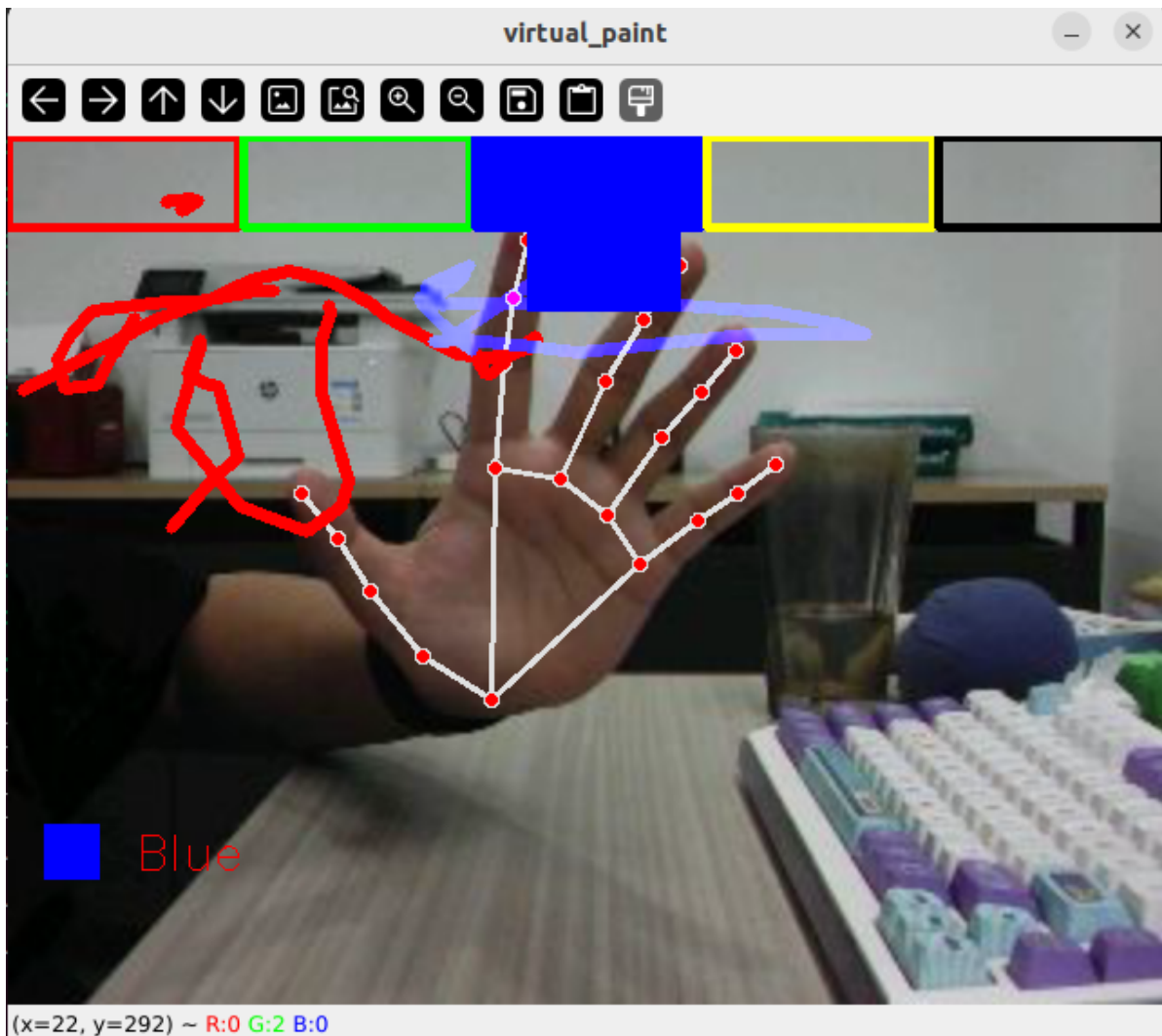
4. Operating Instructions

- **Draw:** extend the index finger and move the fingertip across the frame to draw lines
- **Select color:** touch the color bar at the top of the frame with the index fingertip to switch colors
- **Erase:** select Eraser or overwrite with a specific color
- Press `q` to exit

4. Observed Results

- As the index fingertip moves across the frame, continuous colored lines are left on the canvas
- The color bar at the top of the frame provides multiple color choices
- The drawing canvas is overlaid on the original image





5. Code Walkthrough

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/virtual_paint.launch.py`
- Node:
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/VirtualPaint.py`

1. Core Node Analysis

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/VirtualPaint.py
class VirtualPaint(Node):
    def __init__(self):
        super().__init__("virtual_paint_node")
        self.mp_hand = mp.solutions.hands
        self.hands = self.mp_hand.Hands(
            static_image_mode=False, max_num_hands=1,
            min_detection_confidence=0.5, min_tracking_confidence=0.5)
        # Canvas initialization
        self.canvas = None
        # Color list + top color bar
        self.colors = [(255,0,0), (0,255,0), (0,0,255), ...]
```

```

self.current_color = self.colors[0]
self.prev_pos = None

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    if self.canvas is None:
        self.canvas = np.zeros_like(frame)
    frame = cv.flip(frame, 1) # mirror the frame for hand-eye coordination

    # Draw the top color selection bar
    for i, color in enumerate(self.colors):
        cv.rectangle(frame, (i*80, 0), ((i+1)*80, 60), color, -1)

    results = self.hands.process(cv.cvtColor(frame, cv.COLOR_BGR2RGB))
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            # Index fingertip = landmark 8
            cx = int(hand_landmarks.landmark[8].x * frame.shape[1])
            cy = int(hand_landmarks.landmark[8].y * frame.shape[0])
            # Color selection area (y < 60)
            if cy < 60:
                self.current_color = self.colors[cx // 80]
            else:
                if self.prev_pos:
                    cv.line(self.canvas, self.prev_pos, (cx, cy),
                           self.current_color, 10)
                    self.prev_pos = (cx, cy)
        frame = cv.addweighted(frame, 0.5, self.canvas, 0.5, 0)
        cv.imshow(self.window_name, frame)

```

Key logic:

- `cv.flip(frame, 1)`: mirrors the frame horizontally so hand movement matches intuition (hand-eye coordination)
- Index fingertip = landmark index 8 (the 8th of the 21 points); coordinates are normalized to 0~1 and must be multiplied by the image width/height
- Color selection area $y < 60$: when the hand is raised to the top of the frame it touches the color bar; the color index is mapped from the x coordinate divided by 80
- The canvas is an independent layer `self.canvas`; after drawing lines on the canvas, `cv.addweighted` blends it semi-transparently with the original image

2. Key Parameters

Parameter definition files:

- Launch:


```
~/microros_ws/src/microros_v2_mediapipe_pkg/launch/virtual_paint.launch.py
```
- Node:


```
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/VirtualPaint.py
```

Parameter	Type	Default	Description
<code>detector_con</code>	float	0.5	Detection confidence threshold
<code>track_con</code>	float	0.5	Tracking confidence threshold

3. Node Description

Node	Function
<code>virtual_paint_node</code>	MediaPipe Hands index fingertip tracking + canvas drawing + color selection

6. Frequently Asked Questions

Issue	Cause	Solution
Lines are broken / intermittent	Finger tracking lost or moved too fast	Slow down finger movement and ensure sufficient lighting
Color selection does not work	Fingertip does not touch the color bar accurately	Make sure the index fingertip is in the top area of the frame ($y < 60$)